CryptAlyst — Deep Analysis of the Evaluation Path

Beneath the duplication there are five problems that decide whether any forecast on this platform can be trusted. This document is about those, not about line counts.

Measured from `gpu/backtest_sweep.py`, `internal/api/backtest.go`, `internal/api/super_hybrid.go`, `internal/api/forecast_verify.go`, `internal/api/live_engine.go` at commit `a74fbf9`

D1. The optimiser does not measure skill

```
ss_tot = ((actual - mu) ** 2).sum()          # variance of the FUTURE prices
r2 = 1 - ss_res / torch.clamp(ss_tot, min=1e-12)
...
out.sort(key=lambda r: -r["r2"])             # the winner is chosen by r2
top = out[0]
```

`ss_tot` is the variance of the future prices inside each window. In a window where price moves 5%, that variance is enormous next to a 1% forecast error, so **R^2 lands near 0.96 whichever model produced the path**. In a quiet window the same model scores much lower.

> **Consequence:** the ranking is mostly a measure of how much the window moved. This is why the winner is always `bars 600`, why `vwap`, `vwdrift` and `naive` all sit at 0.95–0.98, and why the board reshuffles between models that are statistically indistinguishable.

> **Fix:** rank by skill against a random walk measured at the anchor, on returns rather than levels; report R^2 of the return series, not of the price series.

D2. An honest metric is computed, then discarded

```
flat_mae = (actual[0, 0] - actual).abs().mean()
skill    = 1 - mae / torch.clamp(flat_mae, min=1e-12)
```

`skill_vs_flat` is the one number that answers the real question — *did this beat doing nothing?* It is written into every result row and never used to select anything.

The baseline is also wrong: `actual[0, 0]` is the **first future bar**, not the anchor price. The "do nothing" benchmark is one step clairvoyant, so the comparison is biased in a direction nobody chose or documented.

> **Consequence:** the platform computes its own falsification test and ignores the answer.

> **Fix:** the baseline is `p[anchor]` held flat; selection requires skill > 0, not merely a high R^2.

D3. Selection over a thousand candidates with no out-of-sample protocol

One sweep scores `models × lookbacks × windows × horizons` — roughly a thousand candidates — sorts, takes the maximum, and reports that maximum **on the same data it was chosen from**. There is no holdout, no penalty for the number of trials, no significance test.

> **Consequence:** the expected best-of-1000 R^2 rises with the candidate count alone, even for pure noise. The engine re-runs this every cycle and keeps the max-ever winner, which compounds the effect over time.

> **Fix:** choose on one span, report on another; require the margin to beat a stated confidence bound before a model is called “best”.

D4. There is no closed loop

Nothing stores a prediction and later compares it with what happened. The only feedback is the same static window, re-scored. Therefore:

- the platform cannot answer *“was the model right yesterday?”*;
- the composite's weights (`1/err²`) are fitted and evaluated on the same window;
- the 10–90% funnel is never checked for coverage out of sample.

> **Consequence:** every number downstream — the board, the ranking, the “best” badge, the cone — inherits an unfalsifiable figure. **This is the finding that makes the others matter:** with a prediction ledger every question becomes answerable; without one, none are.

D5. “Skill” is defined four times, differently

PLACE	COMPUTES	BASELINE
`gpu/backtest_sweep.py`	R^2 on price levels, per window	future-window variance
`internal/api/backtest.go`	its own MAPE / R^2 over anchors	window mean
`internal/api/super_hybrid.go`	weights from $1/err^2$	same window it is fitted on
`internal/api/forecast_verify.go`	coverage and calibration	its own holdout split

> **Consequence:** numbers from the Forecast view and the GPU board are not comparable, and nothing in the code says so. A model can look strong on one screen and weak on another with both being “correct”.

D6. The engine cannot be replayed

`live_engine.go` reads `time.Now()` directly, holds one mutex over a forty-field struct, and records results into `map[string]any`. There is no decision log.

> **Consequence:** “why did it choose `vwdrift lb38` at 21:05?” has no answer six hours later, and no test can drive the engine through a scenario — which is why the test suite passes while covering almost no engine behaviour.

The structural fix: one kernel, one ledger, one policy

```
kernel (pure)
  forecast(series, spec)            -> path
  outcomes(series, anchors, horizon) -> actual
  score(path, actual)               -> { mae, skill_vs_rw, dir_hit, coverage }
        one definition of skill, one baseline (random walk at the anchor)

ledger (append-only)
  prediction{ spec, cid }  ->  outcome{ cid, realised }  ->  score

policy
  select(specs, scores) -> requires skill > 0 at a stated confidence,
                           measured on data the candidate was not chosen from
```

Once those three exist:

1. the sweep ranks by **skill versus random walk**, so `naive` becomes the floor instead of a rival at 0.95;
2. the board can state *“this model beat doing nothing on 63% of its predictions”* — a claim that can be falsified;
3. the funnel's coverage becomes a measured number rather than a drawn cone;
4. the four definitions in D5 collapse into the kernel, and the screens finally agree.

Why this outranks the refactoring list

CHANGE	WHAT IT BUYS
axis fix, 1,167-line split, 30 intervals	a codebase that is easier to change
evaluation kernel + ledger + policy	a platform whose claims can be checked

The cost is small: one scoring module, one append-only ledger, one selection rule. The engine, the GPU runner, the board and the chart keep working — they simply stop each carrying their own definition of “good”.